

RETAILFLOW ANALYTICS PLATFORM

Plateforme data e-commerce : Ingestion GCP, modélisation DBT et visualisation
Streamlit

Documentation du projet - Product Backlog

Version 0.3 - 2026

1. Vision

Construire une plateforme data permettant d'ingérer, transformer et analyser des données e-commerce afin de produire des KPI métier.

2. Product Backlog

Dans ce backlog, deux types de user stories sont distingués :

- User stories métier : exprimées du point de vue des utilisateurs finaux (Data Analyst, utilisateur métier). Elles décrivent des besoins fonctionnels directement liés à l'usage de la plateforme et à la création de valeur métier.
- User stories techniques : exprimées du point de vue du système ou des développeurs. Elles concernent l'ingestion, l'industrialisation et la fiabilité du pipeline, indispensables au bon fonctionnement de la plateforme mais non directement visibles par les utilisateurs.

EPIC 1 - Data Ingestion

ID	User Story	Type	Rôle	Priorité	SP	Status
US1	En tant que Data Engineer, je veux ingérer un fichier CSV dans BigQuery afin de stocker les données brutes.	Technique	Data Engineer	Haute	3	Fait
US2	En tant que Data Engineer, je veux automatiser l'ingestion de plusieurs fichiers afin de construire un pipeline scalable.	Technique	Data Engineer	Haute	5	En cours
US3	En tant que système, je veux gérer les erreurs d'ingestion afin d'assurer la fiabilité du pipeline.	Technique	Système	Moyenne	5	À faire

EPIC 2 - Data Transformation (DBT)

ID	User Story	Type	Rôle	Priorité	SP	Status
US4	En tant que Data Engineer, je veux transformer les données brutes afin de corriger les types et nettoyer les données.	Technique	Data Engineer	Haute	5	À faire
US5	En tant que Data Analyst, je veux des tables métiers afin d'analyser les performances e-commerce.	Métier	Data Analyst	Moyenne	8	À faire

EPIC 3 - Data Visualization

ID	User Story	Type	Rôle	Priorité	SP	Status
US6	En tant qu'utilisateur, je veux visualiser les KPI (CA, commandes, clients) afin de piloter l'activité.	Métier	Utilisateur	Haute	5	À faire

EPIC 4 - Industrialisation

ID	User Story	Type	Rôle	Priorité	SP	Status
US7	En tant que développeur, je veux dockeriser l'application afin de garantir la reproductibilité.	Technique	Développeur	Moyenne	5	À faire
US8	En tant qu'équipe data, je veux automatiser le pipeline via CI/CD.	Technique	Équipe Data	Faible	8	À faire

3. Planification des Sprints (4 semaines)

SPRINT 1 (Semaine 1) : Ingestion de données

Objectif du sprint

Construire et fiabiliser le pipeline d'ingestion Python. À l'issue de ce sprint, les données brutes Olist sont chargées dans BigQuery de manière automatisée et résiliente aux erreurs.

User Stories

- US1 : Ingérer un fichier CSV dans BigQuery
- US2 : Automatiser l'ingestion de plusieurs fichiers
- US3 : Gérer les erreurs d'ingestion

Tâches techniques

ID	Tâche	US liée	Fichier(s)	Est.
T1	Créer la fonction <code>load_csv_to_bq()</code> avec paramétrage du dataset et de la table cible	US1	<code>ingestion/load_to_bigquery.py</code>	2h
T2	Boucler sur les fichiers CSV du dossier <code>data/raw/</code> et appeler <code>load_csv_to_bq()</code> pour chacun	US2	<code>ingestion/load_to_bigquery.py</code>	2h
T3	Ajouter la gestion des erreurs (<code>try/except</code>) et un logger structuré (fichier + console)	US3	<code>ingestion/load_to_bigquery.py</code>	2h
T4	Écrire les tests unitaires de l'ingestion (mock BigQuery client)	US1,2,3	<code>tests/test_ingestion.py</code> (nouveau)	2h
T5	Rédiger le fichier <code>requirements.txt</code> et documenter les prérequis GCP	US1,2,3	<code>requirements.txt</code> , <code>README.md</code>	30min

Livrables

- Pipeline d'ingestion fonctionnel et testé pour tous les fichiers CSV Olist
- Gestion des erreurs avec logs structurés
- Fichier tests/test_ingestion.py avec tests unitaires
- Tous les tests verts - CI green

SPRINT 2 (Semaine 2) : Transformation DBT

Objectif du sprint

Mettre en place la couche de transformation avec DBT. Les données brutes sont nettoyées, typées et agrégées en tables métiers prêtes pour l'analyse.

User Stories

- US4 : Transformer les données brutes (staging)
- US5 : Créer les tables métiers (marts)

Tâches techniques

ID	Tâche	US liée	Fichier(s)	Est.
T6	Créer les modèles de staging DBT (stg_orders, stg_customers, stg_products)	US4	dbt_project/models/staging/	3h
T7	Écrire les tests de qualité DBT sur les colonnes clés (not_null, unique, accepted_values)	US4	dbt_project/models/staging/schema.yml	1h30
T8	Créer les marts DBT (fct_orders, dim_customers, dim_products) avec agrégations KPI	US5	dbt_project/models/marts/	3h
T9	Configurer dbt_project.yml et profiles.yml (connexion BigQuery)	US4,5	dbt_project/dbt_project.yml, profiles.yml	1h

Livrables

- Modèles de staging DBT opérationnels pour les entités principales (commandes, clients, produits)
- Tests de qualité DBT définis et passants (dbt test)
- Tables marts agrégées disponibles dans BigQuery
- Configuration dbt_project.yml et connexion BigQuery validée

SPRINT 3 (Semaine 3) : Dashboard Streamlit

Objectif du sprint

Développer le dashboard Streamlit connecté à BigQuery. Les KPI métier (CA, commandes, clients) sont visualisables et filtrables depuis l'interface.

User Stories

- US6 : Visualiser les KPI (CA, commandes, clients)

Tâches techniques

ID	Tâche	US liée	Fichier(s)	Est.
T10	Créer la page principale Streamlit affichant le CA total, le nombre de commandes et de clients	US6	streamlit_app/app.py (nouveau)	3h
T11	Ajouter un graphique évolution du CA par mois (requête BigQuery → dataframe → Altair/Plotly)	US6	streamlit_app/app.py	2h
T12	Créer le module de connexion BigQuery réutilisable (client + cache @st.cache_data)	US6	streamlit_app/bigquery_client.py (nouveau)	1h
T13	Ajouter un filtre de période (date_input) pour filtrer les KPI dynamiquement	US6	streamlit_app/app.py	1h

Livrables

- Dashboard Streamlit affichant les KPI depuis BigQuery
- Graphique d'évolution mensuelle du CA
- Filtre de période interactif
- Module de connexion BigQuery réutilisable avec cache

SPRINT 4 (Semaine 4) : Industrialisation

Objectif du sprint

Conteneuriser les services avec Docker Compose et automatiser la chaîne de qualité avec GitHub Actions. La plateforme est reproductible et le pipeline CI/CD actif.

User Stories

- US7 : Dockeriser l'application
- US8 : Automatiser le pipeline via CI/CD

Tâches techniques

ID	Tâche	US liée	Fichier(s)	Est.
T14	Écrire le Dockerfile pour chaque service (ingestion, streamlit_app)	US7	ingestion/Dockerfile, streamlit_app/Dockerfile	2h
T15	Créer docker-compose.yml orchestrant les services ingestion et streamlit	US7	docker-compose.yml (nouveau)	1h30
T16	Configurer la variable d'env GOOGLE_APPLICATION_CREDENTIALS dans Docker Compose	US7	docker-compose.yml, .env.example (nouveau)	30min
T17	Écrire le workflow GitHub Actions : lint, tests, build Docker image à chaque push	US8	.github/workflows/ci.yml (nouveau)	2h
T18	Ajouter les secrets GCP dans les paramètres du repo GitHub (SA key en base64)	US8	.github/workflows/ci.yml	30min

Livrables

- Dockerfiles pour les services ingestion et Streamlit
- docker-compose.yml orchestrant l'ensemble de la plateforme
- Workflow GitHub Actions actif (lint, tests, build)
- Tous les tests verts - CI green

4. Architecture technique

Couche	Technologie	Rôle
Ingestion	Python 3.12 + google-cloud-bigquery	Chargement des fichiers CSV Olist vers BigQuery (couche raw)
Entrepôt de données	BigQuery (GCP)	Stockage des données brutes (raw) et des tables transformées (staging, marts)
Transformation	DBT Core	Modélisation en couches (staging → marts), tests de qualité et documentation des modèles
Visualisation	Streamlit	Dashboard interactif exposant les KPI métier (CA, commandes, clients) depuis BigQuery
Conteneurisation	Docker / Docker Compose	Isolation des services ingestion et Streamlit, reproductibilité de l'environnement
Intégration continue	GitHub Actions	Lint, tests unitaires et build Docker automatisés à chaque push
Authentification GCP	Service Account (JSON)	Accès BigQuery via GOOGLE_APPLICATION_CREDENTIALS, clé injectée en variable d'environnement

Pipeline de données

CSV (Olist) → Ingestion Python → BigQuery raw → DBT staging → DBT marts → Streamlit

Le pipeline suit une architecture ELT (Extract-Load-Transform) : les données brutes sont d'abord chargées telles quelles dans BigQuery (couche raw), puis transformées en place par DBT (staging pour le nettoyage, marts pour les agrégations métier). Streamlit consomme directement les marts via le client BigQuery Python.

Séparation des couches BigQuery

- raw : données brutes chargées directement depuis les CSV, sans transformation
- staging : données nettoyées et typées (corrections de types, renommages, filtres qualité)
- marts : tables agrégées et orientées métier consommés par Streamlit et les analyses